

strip 스펙트로그램 띠

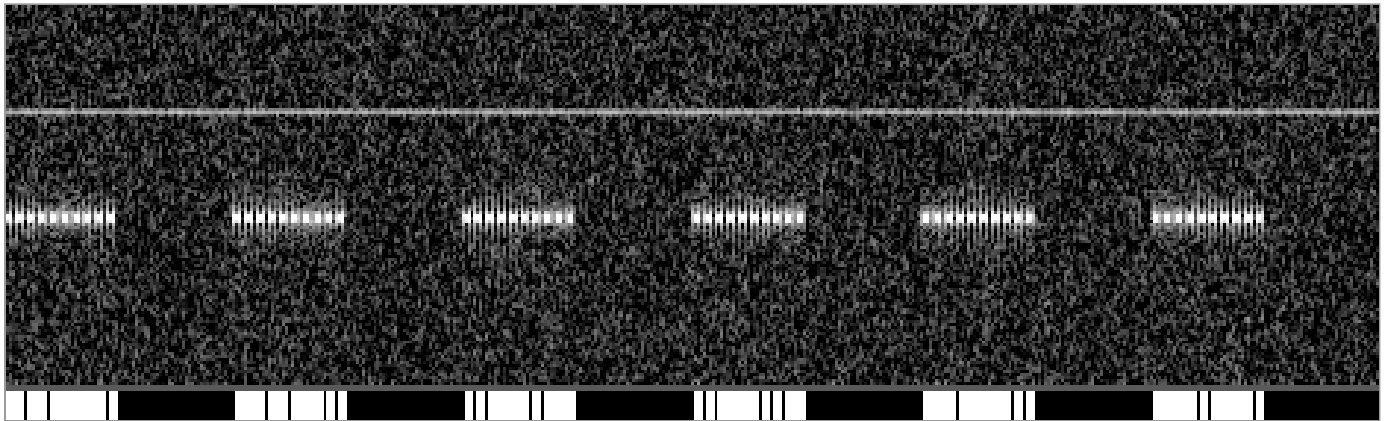
GUI(FFT 256 · Hamming · 0~fs/2)와 같은 표시로 같은 파일을 그린다 — 디코드가 같은지 눈으로 판정하는 결정 실험

코드 72줄 · 총 3쪽 | 2026-08-19 · 5624906+수정중 | 출력은 PNG 파일 — 아무 이미지 뷰어로 연다 (matplotlib 불필요)

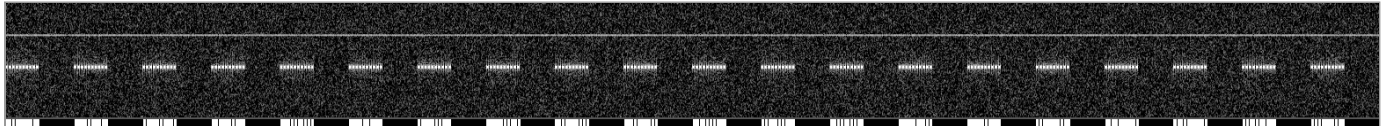
- ① 이 코드를 **signus/** 옆에 **strip.py** 로 저장한다.
- ② **python3 strip.py kat** — 출력 한 줄을 아래와 글자까지 비교하고, 생성된 **strip-kat.png** 를 열어 아래 기대 그림과 모양을 비교한다 (다른 필사 오타).
- ③ **python3 strip.py <캡처파일> — <캡처파일>.png** 가 생긴다. GUI 로 같은 파일을 연 화면과 나란히 비교한다.
- ④ 회신 세 가지: (가) strip 출력 한 줄(#코드까지) (나) PNG 가 아래 두 기준 그림 중 어느 부류인지 한 마디 (다) GUI 가 표시하는 파일 길이(초) — 출력의 **s** 값과 다른 샘플 해석(비트수/채널)이 다른 것이다.

```
strip kat n60000 f10000 s6.0 px467x141 lo-43 rg27 fb28 #thkp
```

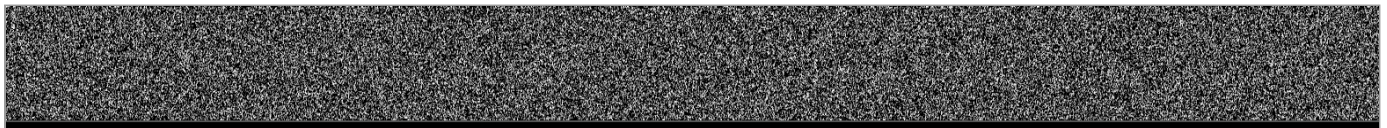
kat 기대 그림 — 장비의 strip-kat.png 가 이와 같아야 한다



버스트가 있는 캡처의 예 — GUI 에 신호가 보인다면 이 부류



잡음뿐인 캡처 — 지금 signus 가 읽은 결과는 이 모습을 예언한다



```

1  """strip.py - 캡처를 GUI 와 같은 방식(FFT 256 · Hamming · 50% 겹침 · 0~fs/2)의 흑백
2  스펙트로그램 띠 PNG 로 그린다 (장비 필사용, signus/ 옆에 저장).
3  그림 맨 아래 흰 띠 = find_bursts 가 "신호 있음" 으로 판정한 구간 - 위(실제)와 눈으로 대조.
4      python3 strip.py kat          # 자기검증: strip-kat.png 생성 + 출력 줄을 표지와 대조
5      python3 strip.py <캡처파일>   # <캡처파일>.png 생성 - GUI 와 나란히 비교
6  """
7  import struct
8  import sys
9  import zlib
10
11 import numpy as np
12 from scipy.signal import stft
13
14 from signus import dsp, sigio
15 from signus.cli import check_code
16
17
18 def png_gray(img, path):
19     img = np.clip(img, 0, 255).astype(np.uint8)
20     h, w = img.shape
21
22     def chunk(tag, data):
23         c = tag + data
24         return struct.pack(">I", len(data)) + c + struct.pack(">I", zlib.crc32(c))
25
26     raw = b"".join(b"\x00" + img[r].tobytes() for r in range(h))
27     with open(path, "wb") as fh:
28         fh.write(b"\x89PNG\r\n\x1a\n"
29                 + chunk(b"IHDR", struct.pack(">IIBBBBB", w, h, 8, 0, 0, 0, 0))
30                 + chunk(b>IDAT", zlib.compress(raw, 6)) + chunk(b"IEND", b""))
31
32
33 if len(sys.argv) < 2:
34     raise SystemExit("사용법: python3 strip.py kat | python3 strip.py <캡처파일>")
35 arg = sys.argv[1]
36 if arg == "kat":
37     fs, n = 10000.0, 60000
38     t = np.arange(n) / fs
39     rng = np.random.default_rng(7)          # 시드 고정 -- numpy 가 재현을 보장한다
40     key = ((np.arange(n) // 250) % 2 == 0) & ((np.arange(n) // 5000) % 2 == 0)
41     x = np.sin(2 * np.pi * 2200 * t) * key + 0.12 * np.sin(2 * np.pi * 3600 * t)
42     x = x + 0.18 * rng.standard_normal(n)
43     name = "strip-kat"
44 else:
45     x, meta = sigio.read(arg)
46     fs = meta.fs
47     x = x.real if np.iscomplexobj(x) else x    # 실수 파형 기준 -- GUI 와 같은 입장
48     name = arg
49     _, _, z = stft(x, fs=fs, window="hamming", nperseg=256, noverlap=128,
50                   return_onesided=True, boundary=None, padded=False)
51     db = 10 * np.log10(np.abs(z) ** 2 + 1e-12)
52     lo = float(np.percentile(db, 25))          # 바닥 근처를 검정으로
53     rg = float(max(10.0, min(35.0, np.percentile(db, 99.5) - lo))) # 자동 대비 -- 고정 35dB 는
54     img = np.clip((db - lo) / rg, 0, 1)[::-1]   # 6dB 급 차이를 씻어내 GUI 와 비교가 안 된다
55     # (아래=0Hz, 위=fs/2 = GUI 의 Y MAX)
56     k = max(1, img.shape[1] // 4000)          # 폭 ~4000픽셀 제한: 열 최대값 풀링이라
57     img = img[:, :img.shape[1] // k * k].reshape(img.shape[0], -1, k).max(2) # 버스트는 남는다
58     xa = dsp.analytic(x)                      # 검출은 pipeline 과 같은 전처리로
59     fb = dsp.find_bursts(xa - xa.mean(), fs)
60     full = fb == [(0, x.size)]                # 통짜 = 미검출 폴백 -- 띠를 다 칠하면 오독

```

```

61 lane = np.zeros(img.shape[1])
62 for s0, e0 in ([] if full else fb):
63     lane[(s0 // 128) // k:(e0 - 1) // 128 // k + 1] = 1.0
64 img = np.vstack([img, np.full((2, img.shape[1]), 0.35), np.tile(lane, (10, 1))])
65 png_gray(img * 255, name + ".png")
66 line = (f"strip {'kat' if arg == 'kat' else 'cap'} n{x.size} f{fs:.0f}"
67         f" s{x.size / fs:.1f} px{img.shape[1]}x{img.shape[0]} lo{round(lo)} rg{round(rg)}"
68         f" fb{0 if full else len(fb)}")
69 print(f"{line} #{check_code(line)}")
70 print(f"{name}.png 저장 - 이미지 뷰어로 열어 GUI 와 나란히 비교. 맨 아래 흰 띠 ="
71       " find_bursts 검출 구간(fb0=통짜 미검출). s 값(초)이 GUI 의 파일 길이와 다르면"
72       " 샘플 해석(비트수/채널)이 다른 것이다")

```